

# TECH CHALLENGE

FASE 1  
SOAT 16

Grupo 75

**Guilherme Toniello Vieira**

RM 374658



**Contato**

49 98832-9708



**Linkedin**

[linkedin.com/in/guilherme-toniello](https://www.linkedin.com/in/guilherme-toniello)



**E-mail**

[guitioniello@gmail.com](mailto:guitioniello@gmail.com)

# INTRODUÇÃO

A seguir, você irá encontrar a proposta arquitetural para o sistema de gestão da oficina mecânica.

Nesse documento, você terá uma visão geral sobre:

- DDD Estratégico
- Padrões de arquitetura e design usados
- Estratégias de CI/CD usadas
- Decisões de Arquitetura (ADRs)
- Diagramas: C4 MODEL, HLD, LLD, etc
- entre outros.

A proposta é um trabalho de conclusão da Fase 1 do curso de pós-graduação de “Software Architecture” da FIAP.

## **Simples e funcional.**

A proposta foi elaborada e implementada seguindo as melhores práticas de mercado com bom-senso e parcimônia.

Arquitetura de Software também é, de certa forma, a arte de atingir os objetivos da forma mais simples, barata e com a melhor qualidade possível.

## **Individual, clareza e qualidade técnica.**

Feito individualmente com capricho e, até um certo carinho, espero que você - leitor, revisor, avaliador - entenda as estratégias usadas e goste do projeto, assim como eu.

Boa leitura,

*Guilherme Toniello Vieira*

RM 374658

# IDENTIFICAÇÃO

## GRUPO 75

### Item

### Informação

Integrante

Guilherme Toniello Vieira - RM 374658

Discord

guilherme034080

Turma

SOAT 16 - FIAP

Video Explicativo

Repositório do Projeto

<https://github.com/GuiToniello/tech-challenge-fase-1-soat16-rm374658>

Instruções de Uso

As instruções de execução, configuração e uso do projeto estão disponíveis no arquivo README do repositório.

SonarQube / SonarCloud

[https://sonarcloud.io/project/overview?id=GuiToniello\\_tech-challenge-fase-1-soat16-rm374658](https://sonarcloud.io/project/overview?id=GuiToniello_tech-challenge-fase-1-soat16-rm374658)

Design Approval Sheet

[https://miro.com/app/board/uXjVHVfHuvI=/?share\\_link\\_id=292099340847](https://miro.com/app/board/uXjVHVfHuvI=/?share_link_id=292099340847)

Artefatos no Miro

DDD Estratégico; Event Storming; Domain Storytelling; C4 Model; Mapa de Contextos Delimitados; Linguagem Ubíqua; Diagramas complementares; Etc.

Anexos

Ao final deste documento, encontram-se os relatórios de análise de vulnerabilidades e segurança, incluindo o relatório do OWASP ZAP e demais evidências geradas durante a validação do projeto.

# DDD ESTRATÉGICO

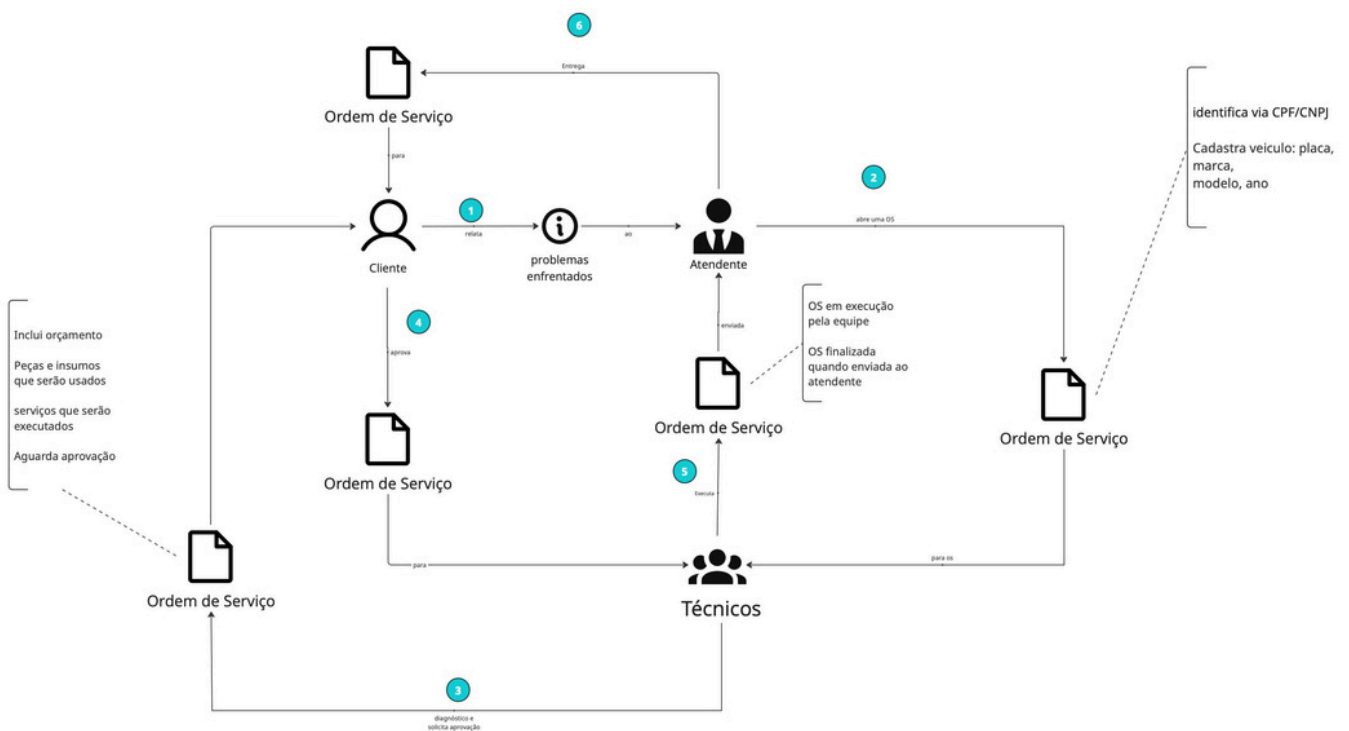
Há um antigo ditado popular que diz:

"Antes de lutar uma batalha, conheça o terreno".

Assim, vamos começar pelo mapeamento dos fluxos de negócio.

Entender como a oficina funciona, seus processos, atores, artefatos e demais envolvidos é fundamental para a proposta arquitetural.

Nesses termos, vamos ao **Domain Storytelling**.



Você pode conferir a versão ampliada no Miro ([link](#))

A imagem acima apresenta o domain storytelling do fluxo principal da oficina.

Os subprocessos, como:

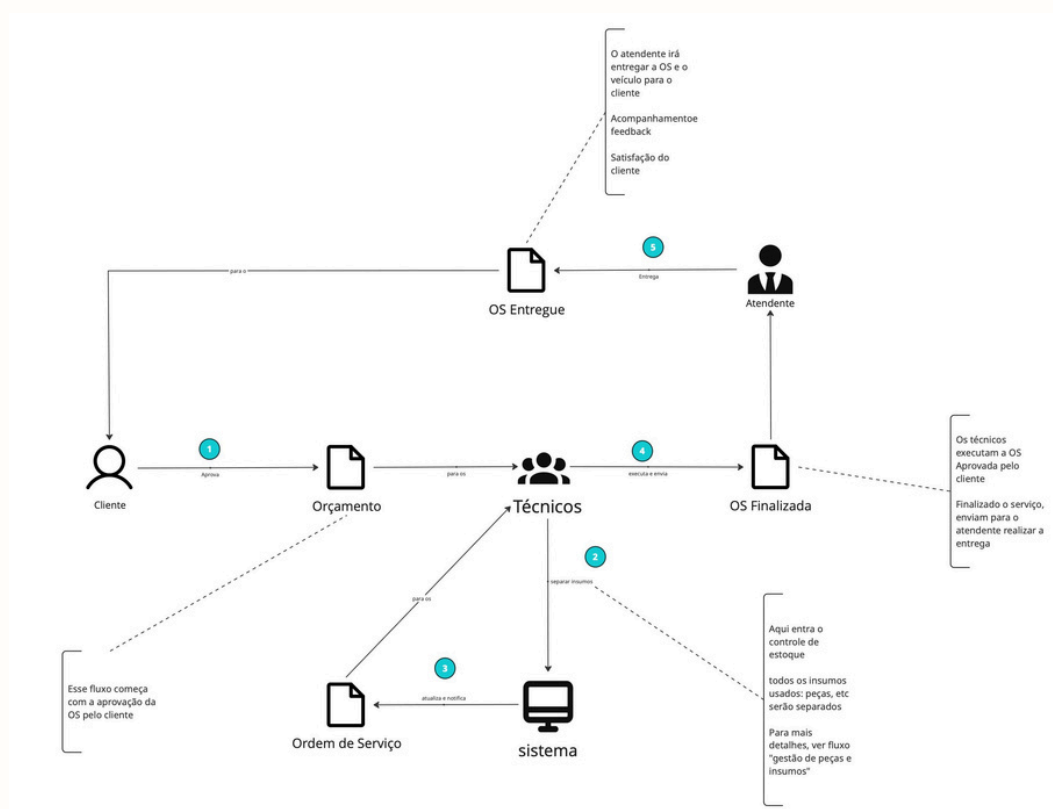
- "Abertura e Orçamento da OS",
- "Execução e Finalização da OS"
- entre outros

serão apresentados a seguir, nas próximas páginas.

Na imagem abaixo, você pode conferir esse mapeamento.



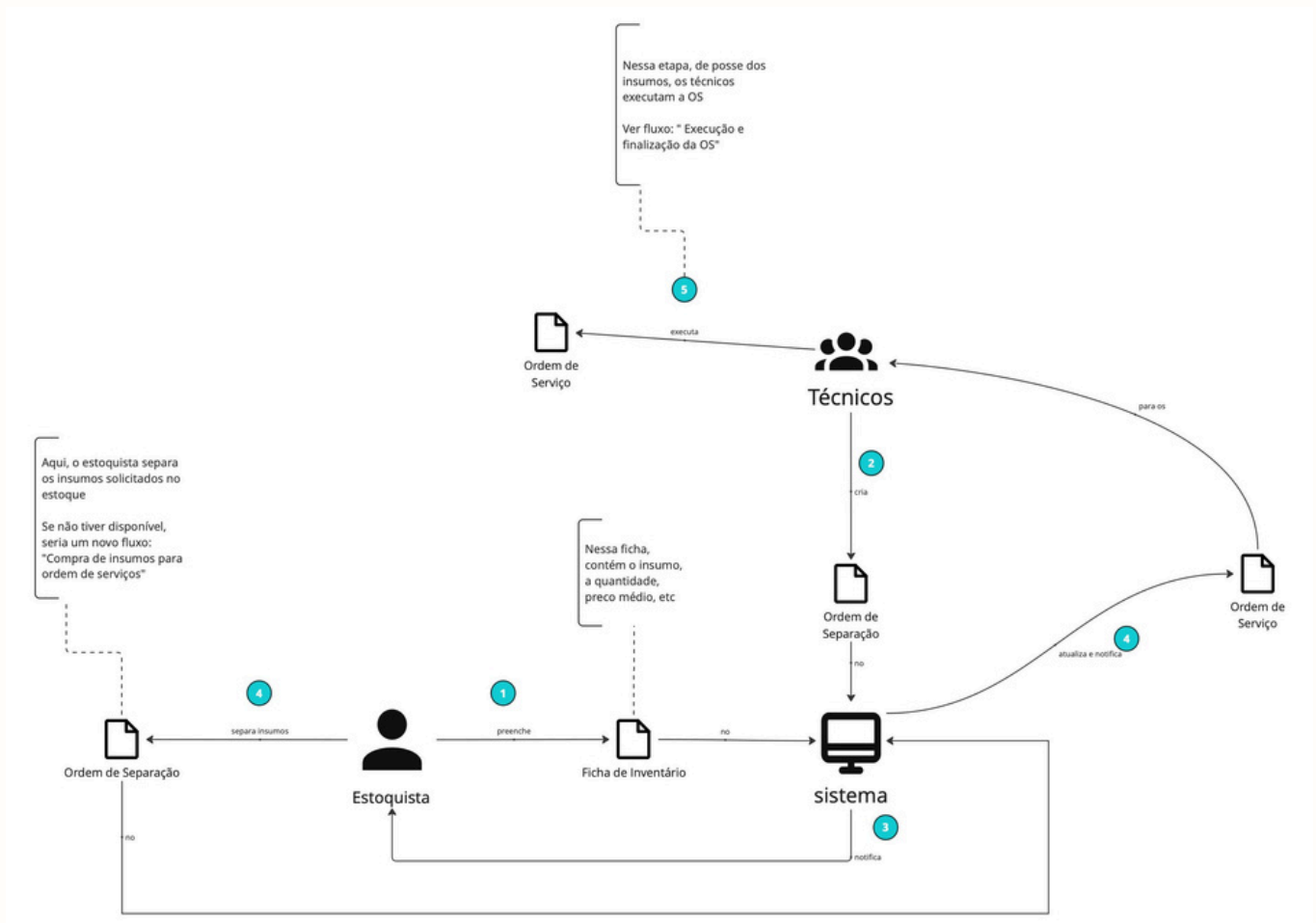
Agora, vamos para o processo de Execução e Finalização da OS.



Você pode conferir a versão ampliada no Miro ([link](#))

Não podemos esquecer do processo de gestão de estoque que explica como funciona o gerenciamento de peças e insumos da oficina.

Assim, abaixo você confere o mapeamento desse importante processo.



Você pode conferir a versão ampliada no Miro ([link](#))

Diante dos diagramas apresentados, podemos concluir que temos o mapeamento suficiente para entender os principais processos de negócio da oficina mecânica.

Agora, precisamos entender passo-a-passo esses fluxos, com mais detalhes, entendendo os eventos, comandos e atores.

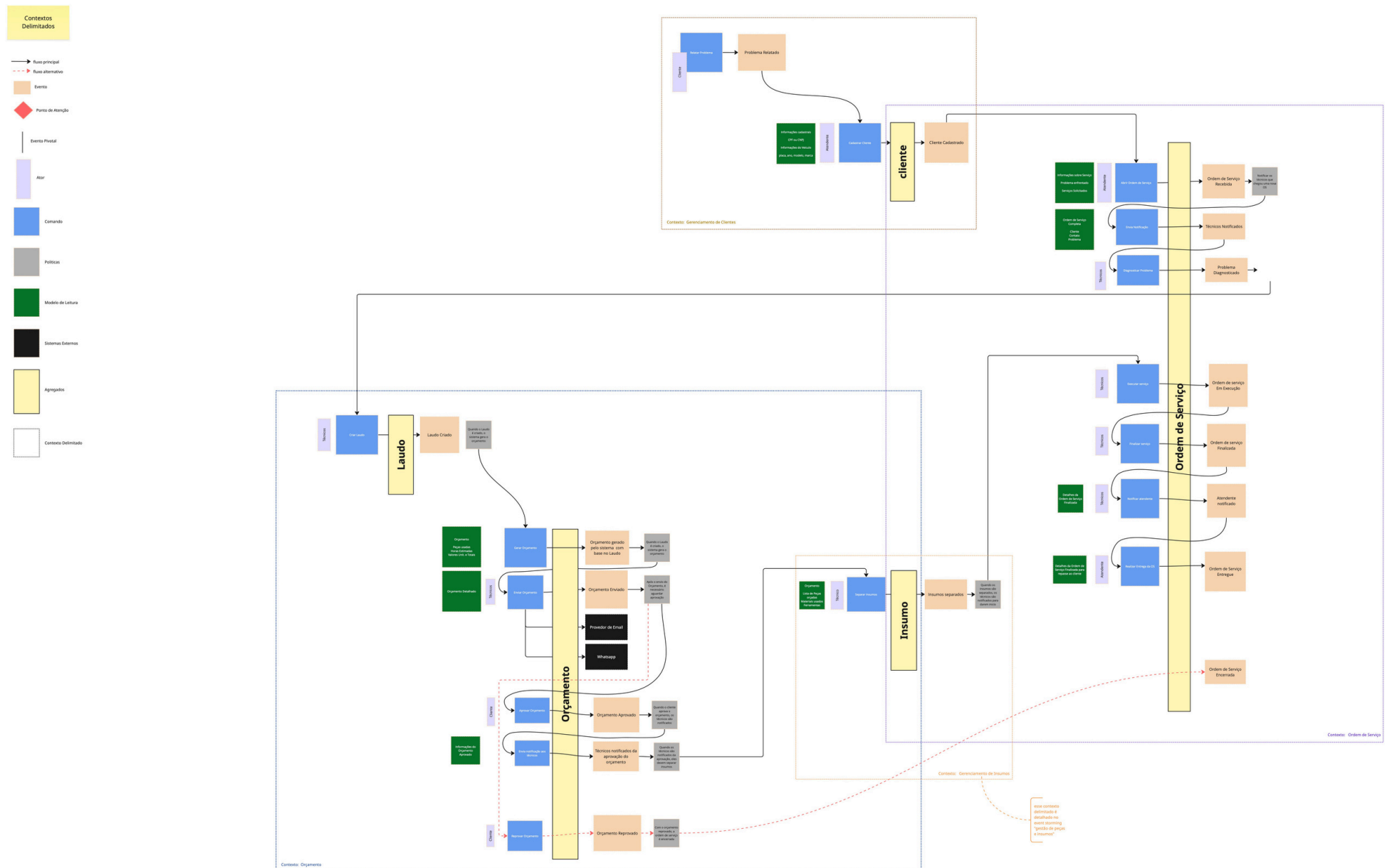
Assim, vamos ao **Event Storming**.

Dessa forma, foi realizado o event storming desse fluxo de negócio.

Abaixo, é apresentado o desenho final do Event Storming de "Abertura e Orçamento da OS".

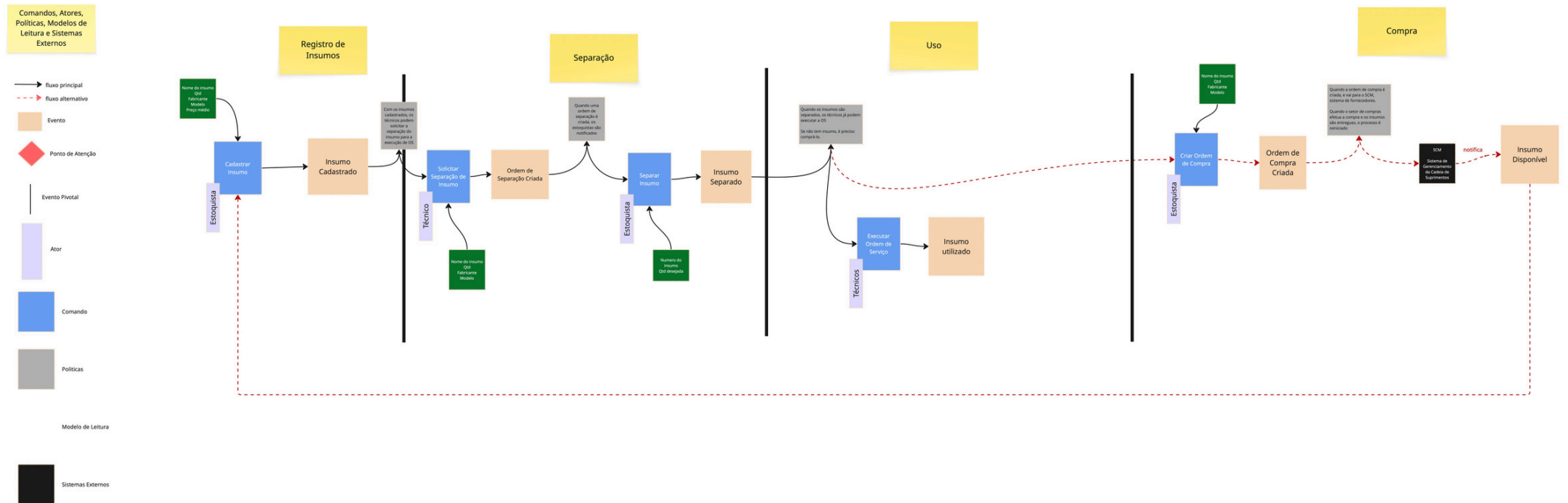


Abaixo, é apresentado o Event Storming de "Abertura e Orçamento da OS" com os agregados e contextos delimitados.



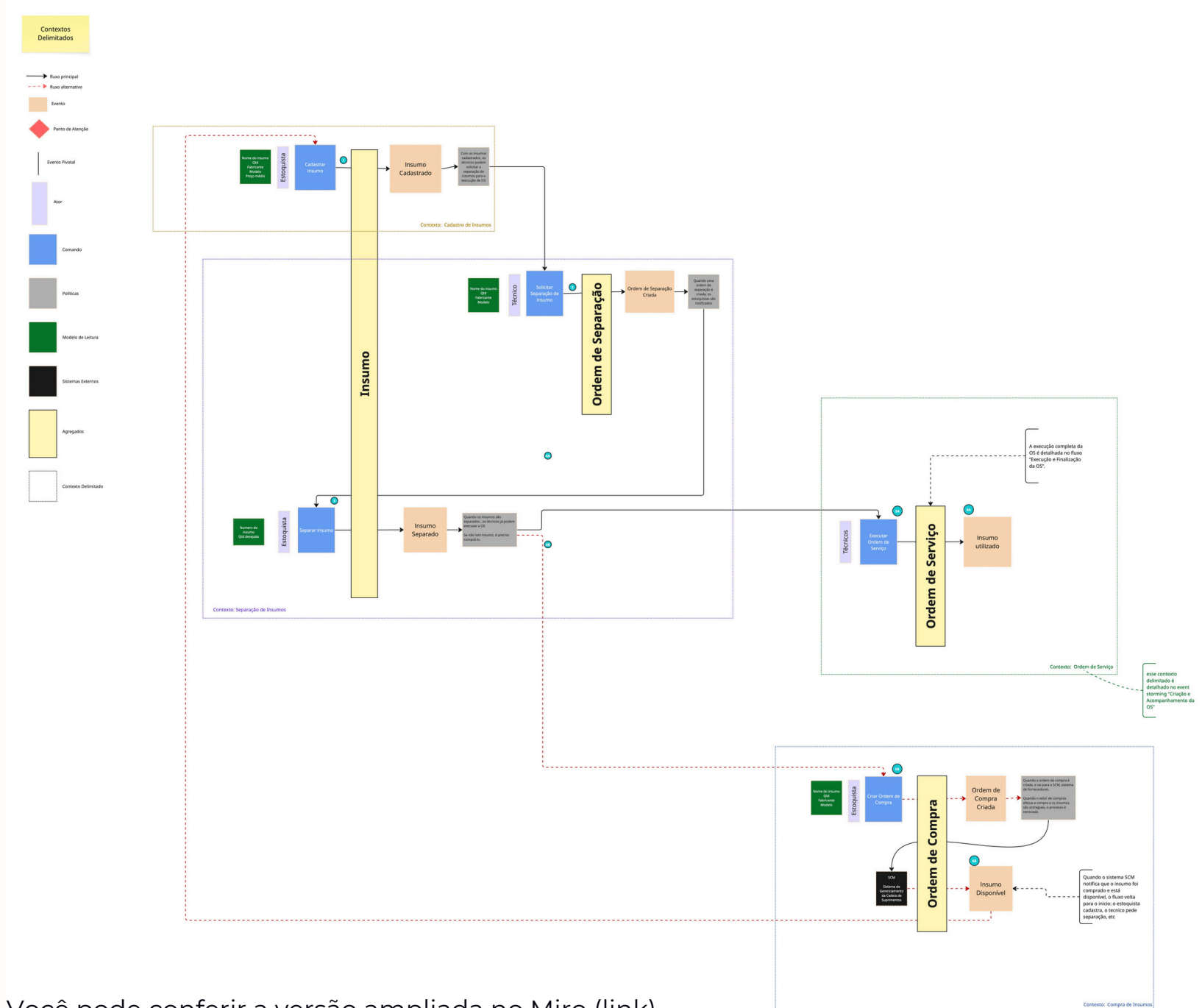
Você pode conferir a versão ampliada no Miro ([link](#))

Logo, o Event Storming de "Execução e Finalização da OS" é apresentado na imagem abaixo.



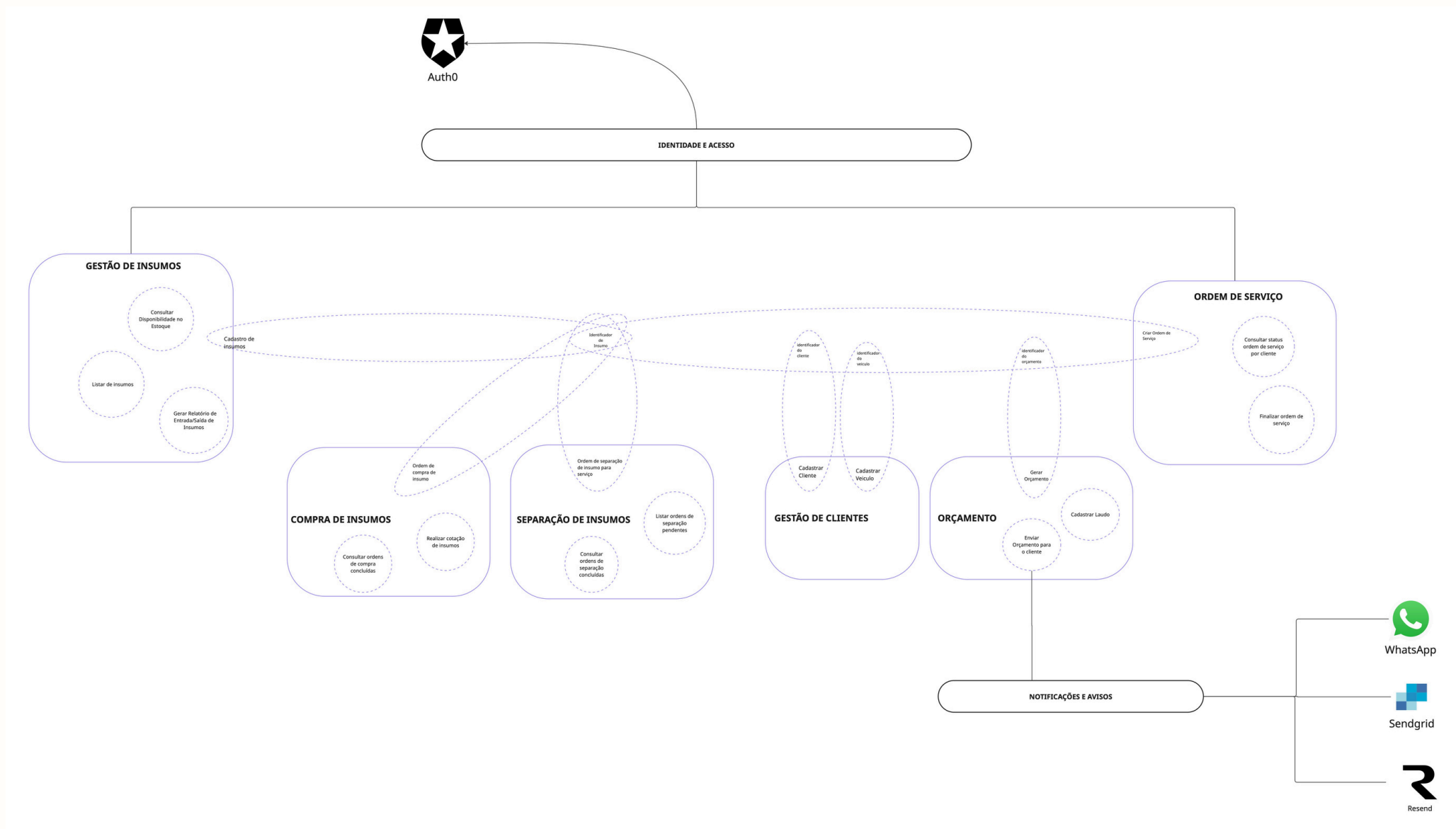
Você pode conferir a versão ampliada no Miro ([link](#))

Em sequência, a imagem abaixo mostra os agregados e contextos delimitados do Event Storming de "Execução e Finalização da OS"



Você pode conferir a versão ampliada no Miro ([link](#))

Por fim, temos o mapa completo de contextos delimitados do sistema, que é apresentado na imagem abaixo.



Você pode conferir a versão ampliada no Miro ([link](#))

Diante de tudo que foi apresentado, podemos concluir que foi realizado um mapeamento robusto dos fluxos de negócio da oficina.

O objetivo do DDD Estratégico não é ser um guia exaustivo com cada mínimo detalhe mas sim ser um mapa geral do negócio e seus principais fluxos.

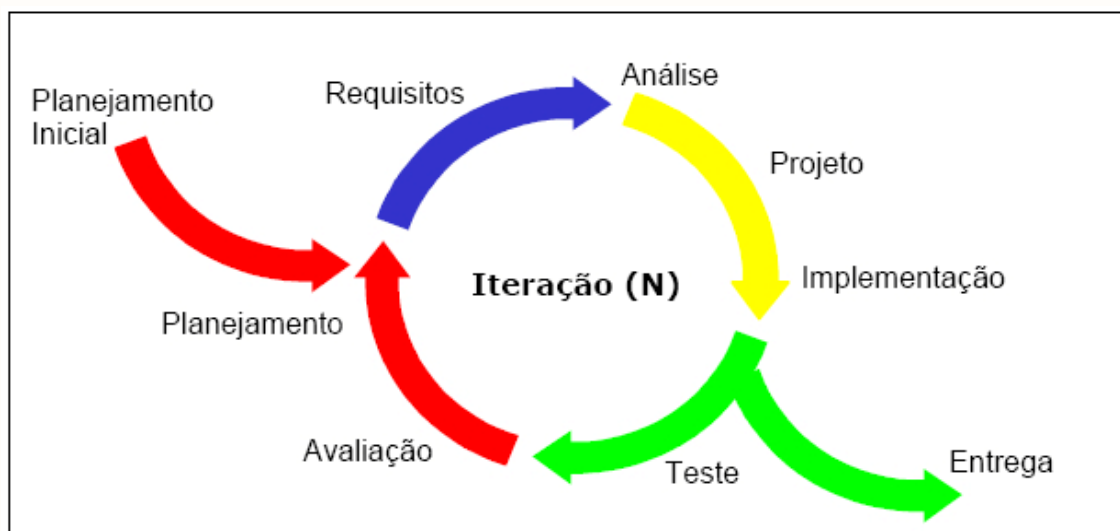
Conectando com as metodologias ágeis que tem por princípio em seu manifesto:

- "Pessoas & suas Interações" mais do que "Processos & Ferramentas"
- "Colaboração com o Cliente" mais do que "Negociar Contratos"

Assim, à luz desses princípios, temos um guia geral e robusto da oficina e seus processos.

No decorrer do desenvolvimento, é natural que exista um aprofundamento e modificação de alguns mapeamentos realizados.

Todavia, os métodos ágeis e a participação dos stakeholders durante o processo de desenvolvimento, iterativo e incremental, nos ajudam a ser ainda mais assertivos.



Dito isso, temos insumos suficientes para projetar o sistema, usando arquitetura evolucionária.

Na próxima seção, iremos nos aprofundar na arquitetura de software que foi usada: padrões, práticas e princípios.

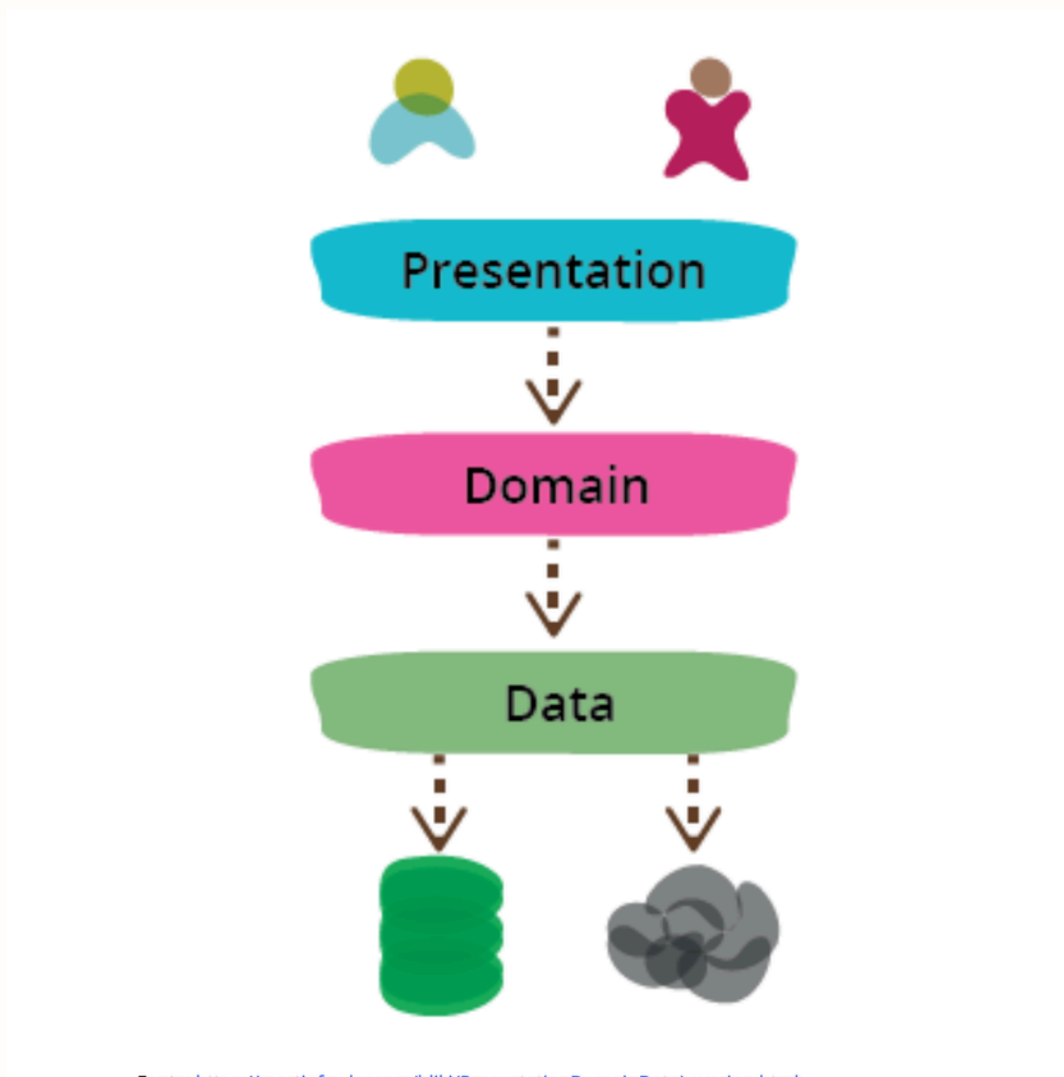
# ARQUITETURA

O projeto será concebido em uma arquitetura que tem como base a arquitetura em camadas: **Layered Architecture**.

A referência bibliográfica usada para esse modelo:

Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). Pattern-Oriented Software Architecture: A System of Patterns.

Além disso, apresento abaixo a imagem da arquitetura em camadas proposta por Martin FOWLER em "Domain Data Layering":



Fonte: <https://martinfowler.com/bliki/PresentationDomainDataLayering.html>

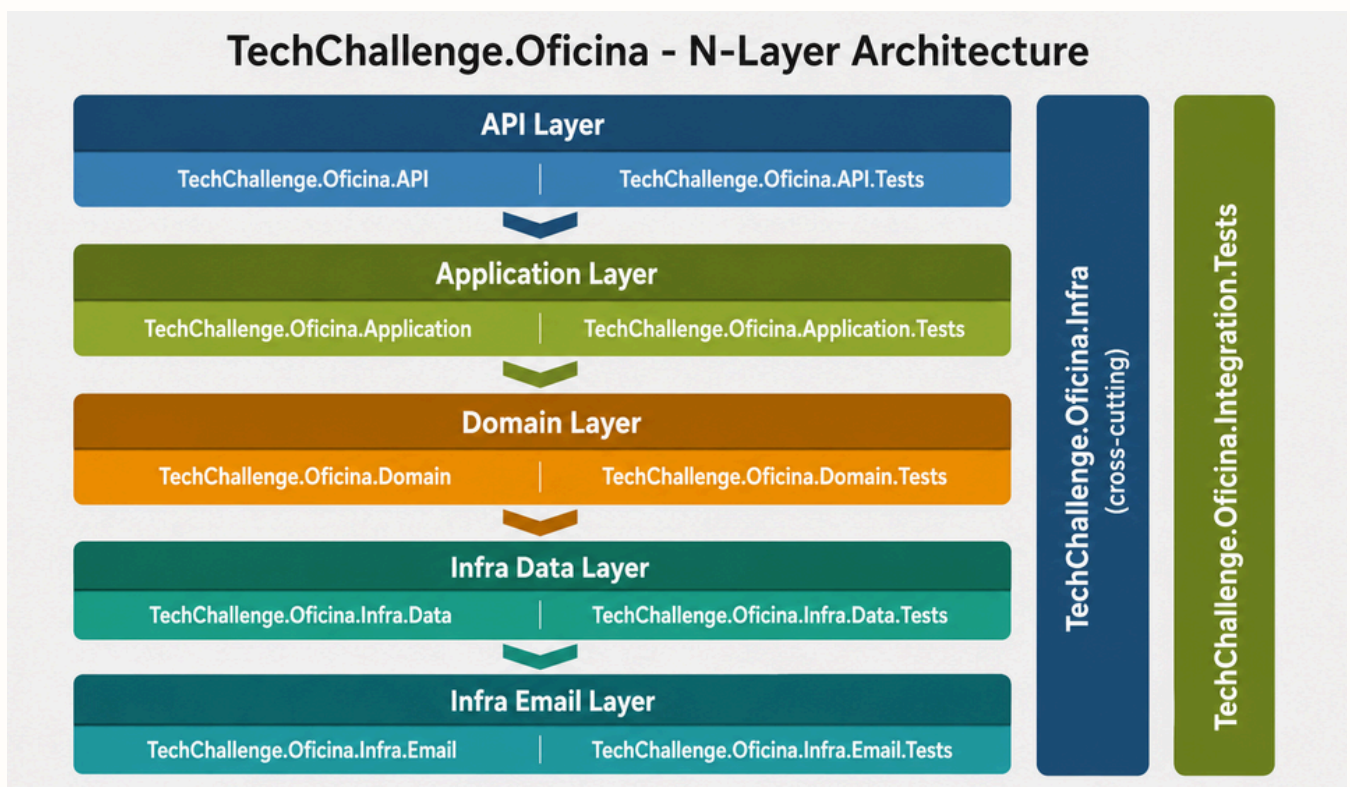
## .Simples e funcional.

Esse estilo arquitetural isola as responsabilidades em cada camada.

Sim, **há trade-offs e desafios conhecidos**.

Para o cenário proposto, é o modelo mais simples e eficaz, é um padrão já conhecido na indústria de software, que atende o cenário.

Na imagem abaixo, você confere uma visão geral sobre as camadas do projeto



.Assim, cada camada é responsável por implementar uma responsabilidade do sistema.

Dessa forma, podemos respeitar o conceito de Separation of Concerns (SoC) na arquitetura.

E também, pavimenta o caminho para que o código utilize o Single Responsibility Principle (SRP), diante do fato que o código de cada camada tem apenas uma responsabilidade, evitando misturar funcionalidades.

Dentro de cada camada, usamos os padrões:

### Padrões Estruturais

- Folder-by-Feature
- Repository Pattern
- Facade Pattern
- Mapper / Adapter Pattern
- Dependency Injection
- Service Layer

### Padrões de Design

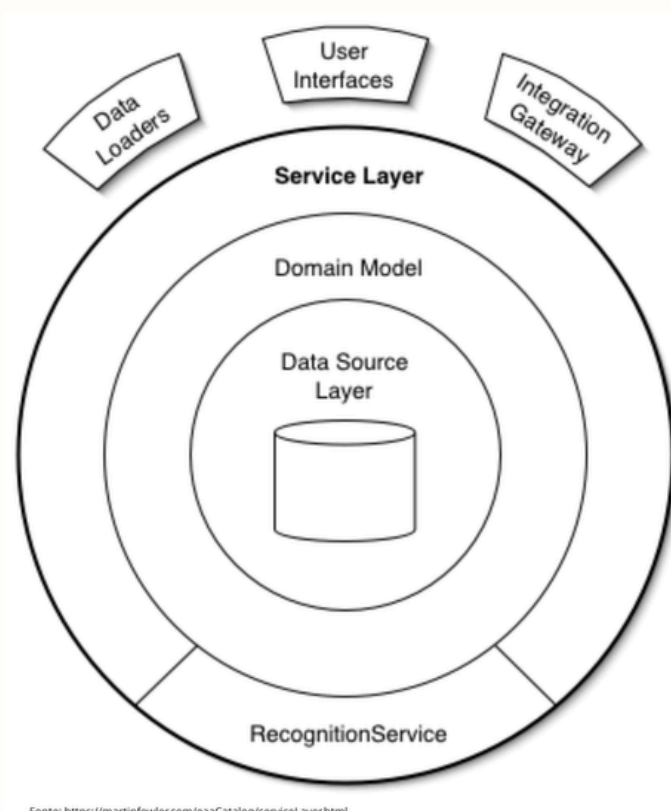
- Factory Method Pattern
- DTO Pattern
- Centralized Exception Handling

### Padrões Comportamentais

- CQRS (Command Query Responsibility Segregation)
  - Feito o nível mais básico: separação da intenção (CQS)
  - Commands (escrita), Queries (leitura), ViewModels (resposta)

Com esses padrões, conseguimos mitigar possíveis problemas da Layered Architecture e conseguimos baixo acoplamento e alta coesão.

Um dos padrões adotados na camada de Application será o Service Layer, conforme imagem abaixo.



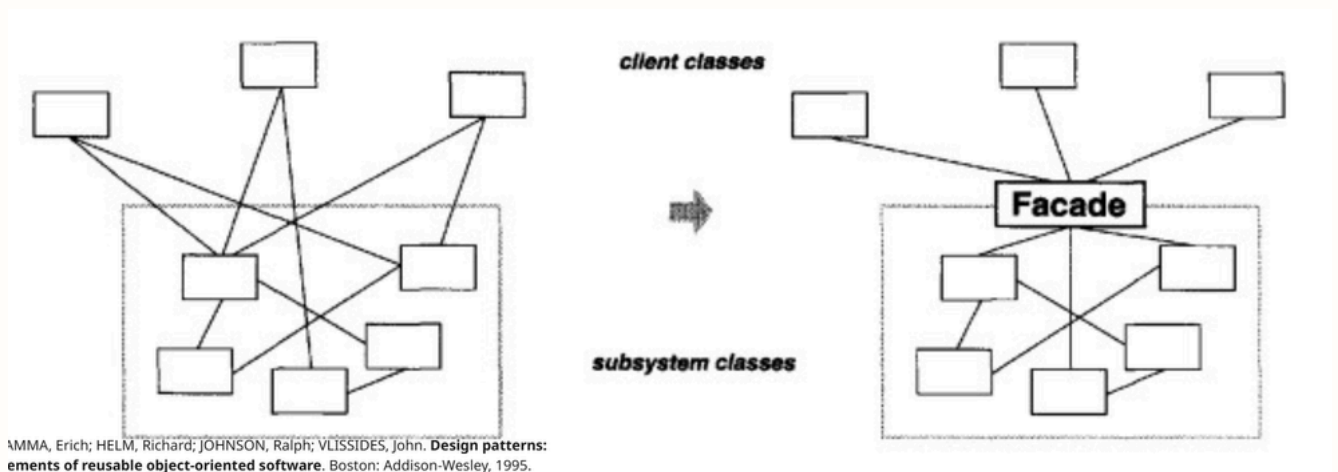
O padrão de **Service Layer**, conforme imagem acima, irá nos ajudar orquestrando o fluxo da aplicação.

Não se engane: as regras de negócio continuam no Domínio, não estamos usando modelo anêmico.

Esses serviços apenas orquestram o fluxo, como enviar email, chamar os repositórios, etc.

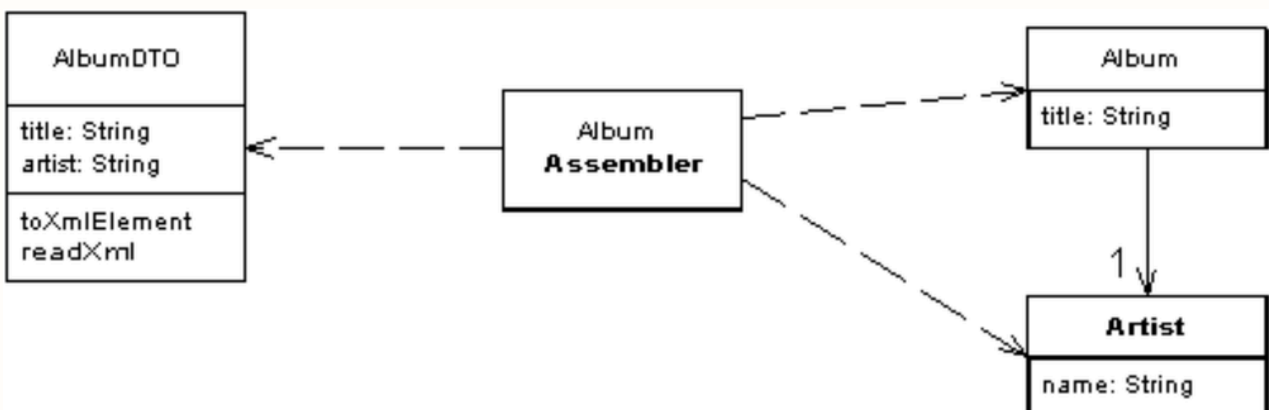
A estratégia de serviços nos ajuda a isolar as regras de negócio do fluxo da aplicação.

Para evitar sobrecarga de propriedades nesses serviços, usaremos também o **Facade Pattern**.



Além disso, para a API, usaremos uma abordagem inspirada no CQRS, em um nível menor, sem separar as bases de dados mas separando a intenção (CQS): Commands, Queries e ViewModel.

Commands, Queries e ViewModels também serão utilizados como DTOs do projeto.



Diante de tudo que foi apresentado aqui na seção de Arquitetura, podemos concluir que temos robusta base teórica para seguir com a implementação.

Com base no que foi apresentado até aqui, é evidente que temos forte base teórica e norteadores claros para seguir com a implementação da arquitetura do projeto.

É evidente que, no decorrer do ciclo de vida do software, esses princípios podem ir mutando e serem adaptados a realidade do time de desenvolvimento e do negócio.

Em ambientes de incerteza, a arquitetura evolucionária é uma prática recomendada para evoluir a arquitetura de forma responsável.

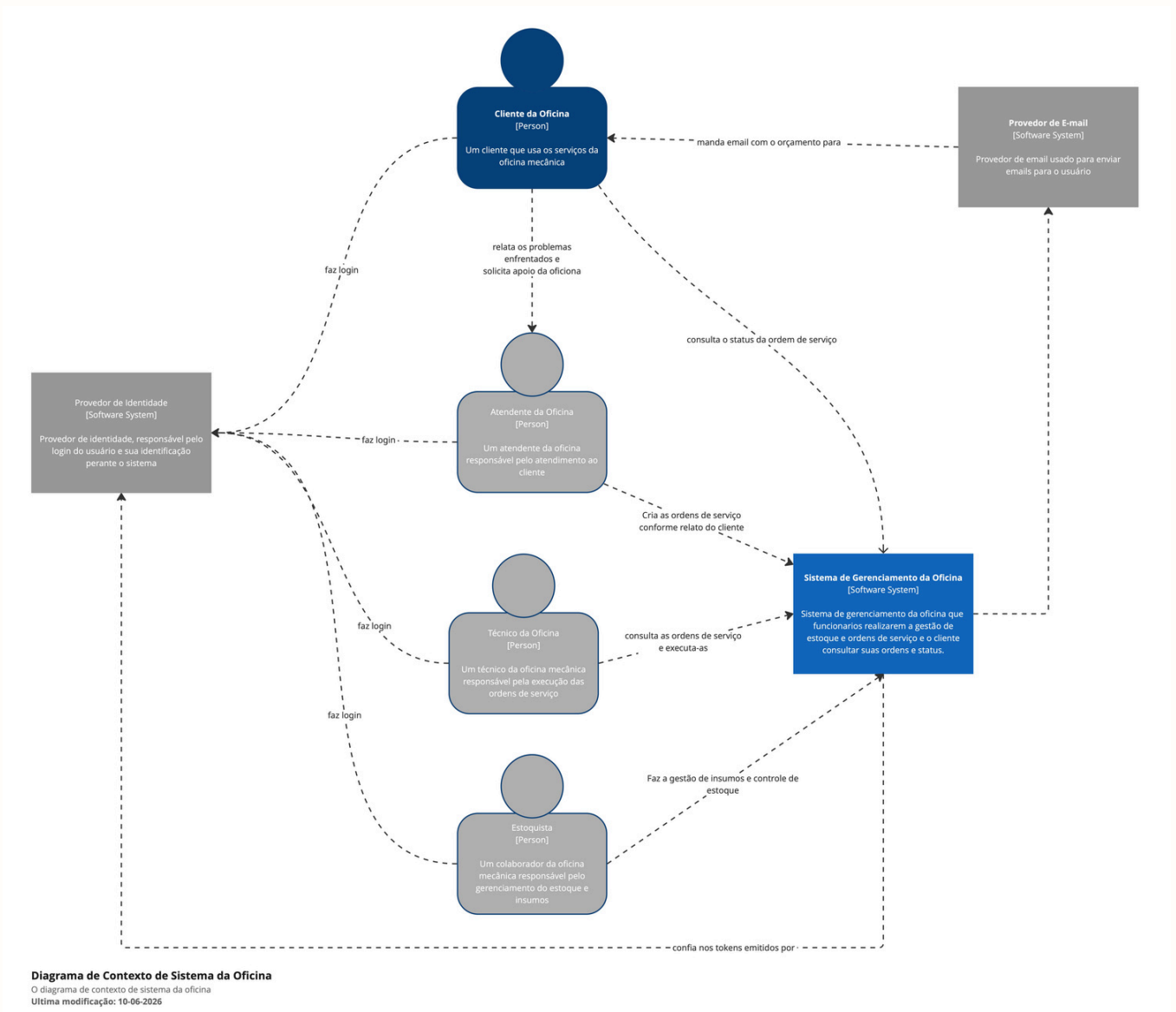
A seguir, serão apresentados os diagramas de C4 Model do sistema implementado.

# C4 MODEL

Dado que, nas seções anteriores, definimos robustamente o DDD Estratégico e estabelecemos nossos princípios e base teórica em Arquitetura, chegou a hora de modelarmos o sistema.

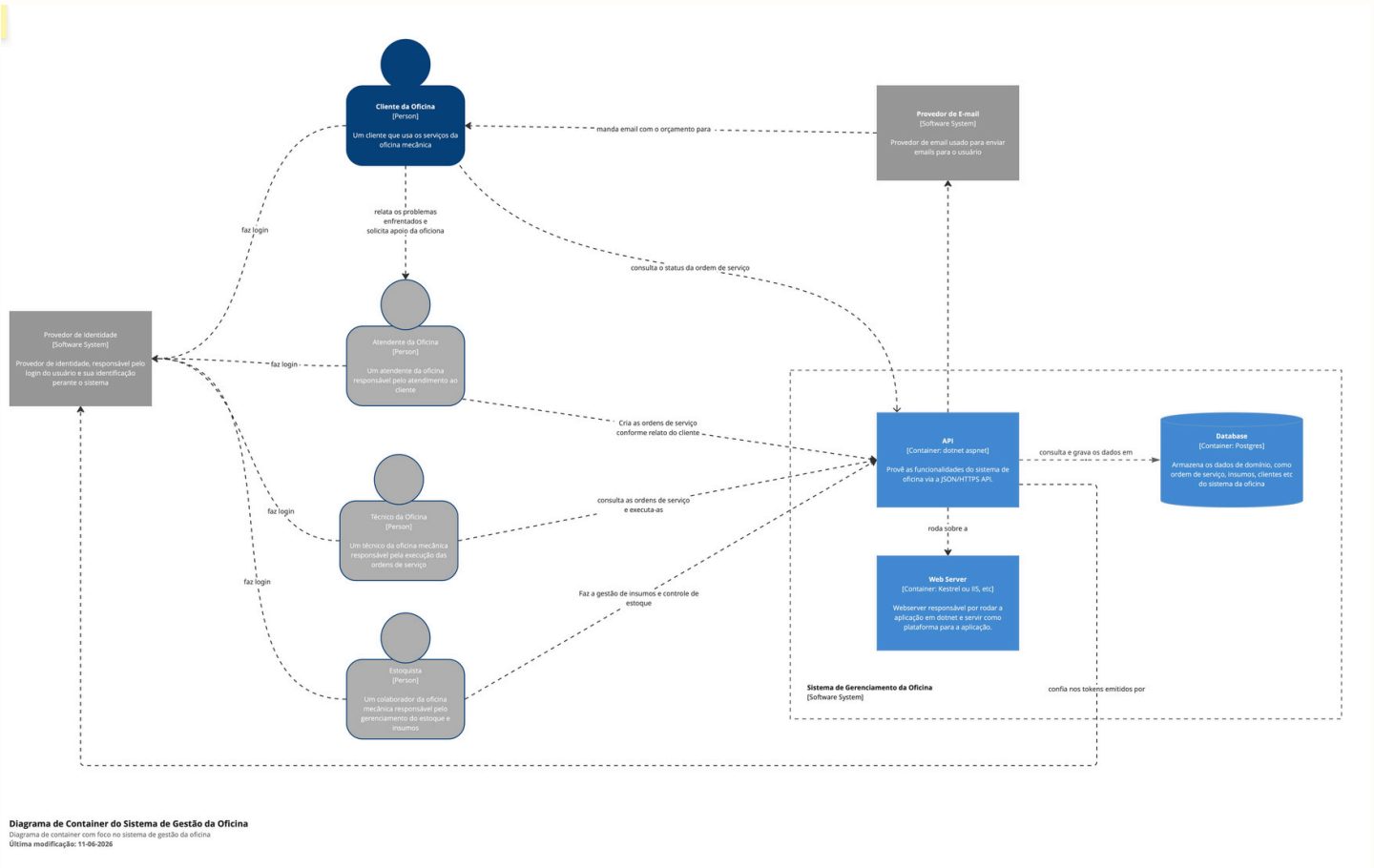
Abaixo, você encontra o primeiro diagrama do C4 MODEL do Sistema de Gestão da Oficina.

Assim, segue o diagrama de **System Context**.



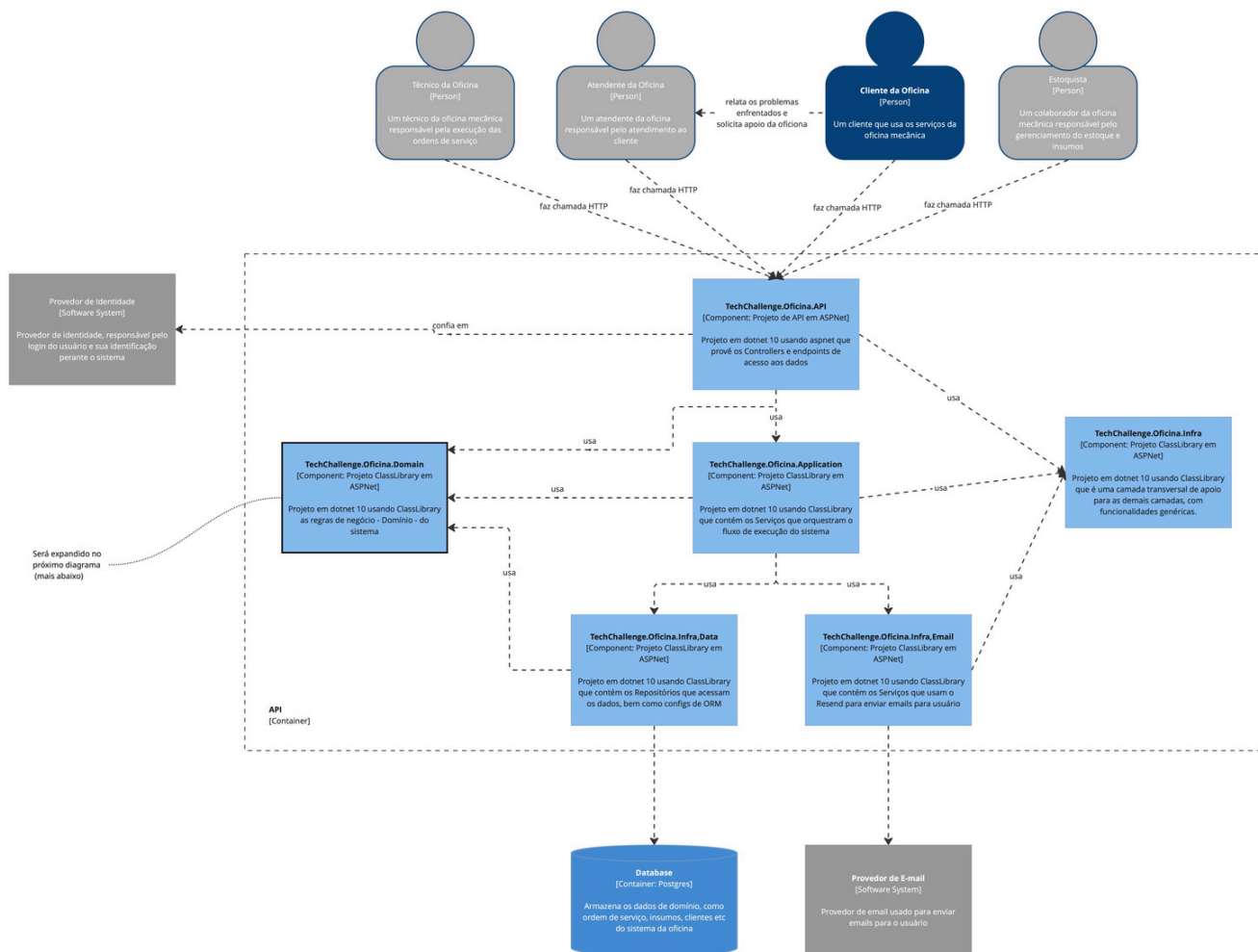
Assim, podemos seguir e expandir o sistema de gestão da oficina para um novo diagrama, detalhando-o mais a fundo.

Segue abaixo do **Container Diagram**



Então, podemos seguir e expandir a API, entendendo melhor como esse importante componente funciona.

Segue abaixo do **Component Diagram**

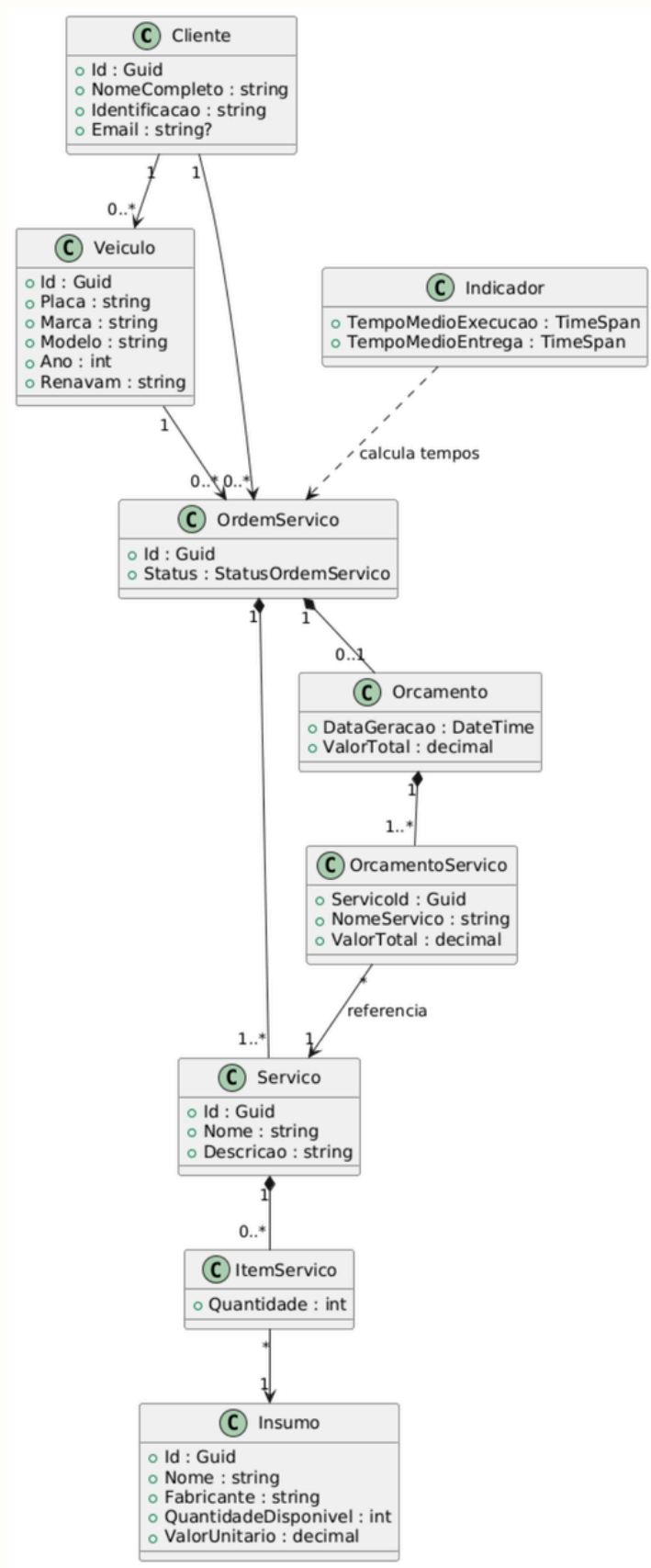


**Diagrama de Componentes para o Sistema de Gestão da Oficina - API**  
O diagrama de componentes para sistema de gestão da oficina com foco na API  
Última modificação: 11-06-2026

Por fim, temos opcionalmente o diagrama de código.

Nessa etapa, vamos nos aprofundar apenas no Domínio, que é o coração do software de acordo com o DDD de Eric Evans.

Segue abaixo do **Code Diagram**



Com todos os diagramas do C4 Model apresentados, é possível concluir que temos agora um planejamento robusto do sistema.

Com esses diagramas, principalmente os 3 primeiros, conseguimos ter uma visão tridimensional sobre o sistema a ser implementado.

Indo além, também expandimos o Domínio, apresentando o diagrama de classes do coração do sistema.

Assim, temos bem esclarecido o sistema que será implementado, suas camadas e responsabilidades.

A partir desse ponto, podemos seguir para a implementação e suas práticas, como Continuous Integration (CI) e Estratégia de Testes Automatizados.

# CI/CD DO PROJETO

A definição das práticas de Continuous Integration (CI) e Continuous Delivery (CD) do projeto são fundamentais para garantir que os requisitos sejam atendidos de forma ágil.

Os requisitos não-funcionais, como segurança, devem estar interligados a processos de CI para prover um maior nível de segurança.

Assim, foi definido que o projeto “Sistema de Gestão da Oficina” tenha um pipeline automatizado para validação das boas práticas de engenharia de software, utilizando o **SonarQube Cloud**.

Além disso, o pipeline também provê feedback de segurança, com análise pelas ferramentas **OWASP Dependency-Check** e **Trivy**.

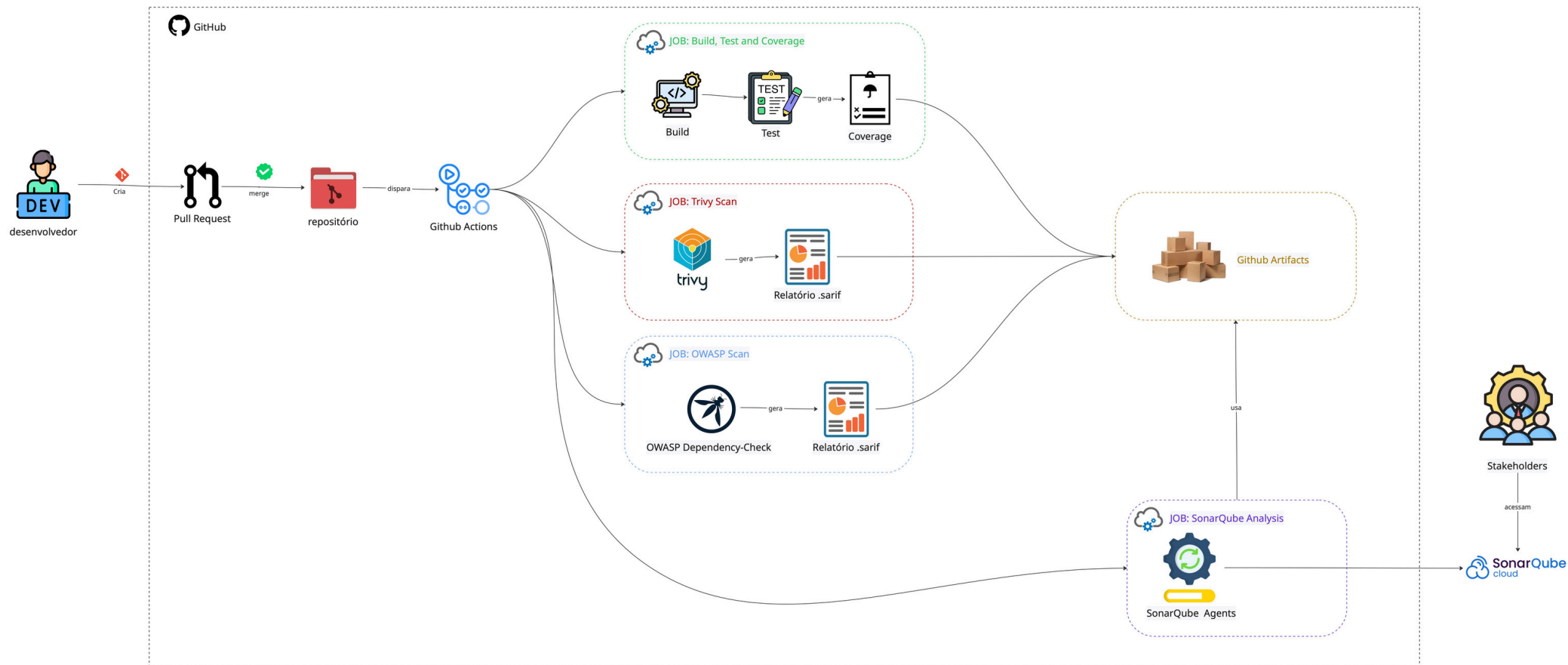
Essas ferramentas validam as dependências do projeto para garantir que o projeto não usa bibliotecas de terceiros com vulnerabilidades conhecidas.

O pipeline também executa os **testes de unidade** e **testes de integração** para garantir que o código que está sendo criado não gerou novos bugs ou cenários adversos.

É gerado também relatórios de **code coverage** que medem a cobertura dos testes automatizados, garantindo uma boa testabilidade para o portal, quando associado aos Quality Gates do SonarQube.

Na próxima página, você pode conferir o diagrama completo de funcionamento do CI do projeto usando **GitHub Actions**.

Todo esse processo foi criado e implementado pensando em **qualidade** e **segurança** para o produto de software que está sendo criado.



Você pode conferir a versão ampliada no Miro ([link](#))

## All workflows

Showing runs from all workflows

🔍 Filter workflow runs

51 workflow runs

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

|                                                                                                                                                                                |      |                                    |     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------------------------------|-----|
| ✓ docs: adicionar link sonar<br>Build, Test, Security Scan and Analyze #52: Commit <a href="#">6db9987</a> pushed by <a href="#">GuiToniello</a>                               | main | 📅 Jun 12, 17:13 GMT-3<br>🕒 11m 3s  | ... |
| ✓ refactor: remover pastas nao usadas da solution<br>Build, Test, Security Scan and Analyze #51: Commit <a href="#">1d51248</a> pushed by <a href="#">GuiToniello</a>          | main | 📅 Jun 12, 17:09 GMT-3<br>🕒 13m 20s | ... |
| ✓ build: adicionar trivy sarif<br>Build, Test, Security Scan and Analyze #50: Commit <a href="#">4936757</a> pushed by <a href="#">GuiToniello</a>                             | main | 📅 Jun 12, 16:56 GMT-3<br>🕒 12m 17s | ... |
| ✓ docs: adicionar seção de execução do projeto<br>Build, Test, Security Scan and Analyze #49: Commit <a href="#">6b0a4a0</a> pushed by <a href="#">GuiToniello</a>             | main | 📅 Jun 12, 16:45 GMT-3<br>🕒 13m 14s | ... |
| ✓ docs: incrementar readme com informações sobre o projeto<br>Build, Test, Security Scan and Analyze #48: Commit <a href="#">248dfa2</a> pushed by <a href="#">GuiToniello</a> | main | 📅 Jun 12, 15:13 GMT-3<br>🕒 10m 53s | ... |
| ✓ build: remover migrations do sonar<br>Build, Test, Security Scan and Analyze #47: Commit <a href="#">65417b5</a> pushed by <a href="#">GuiToniello</a>                       | main | 📅 Jun 11, 18:54 GMT-3<br>🕒 15m 14s | ... |

Você pode conferir a versão ampliada no GitHub ([link](#))

GuiToniello / tech-challenge-fase-1-soat16-rm374658

Code Issues Pull requests Agents Actions Projects Security and quality Insights Settings

Build, Test, Security Scan and Analyze

✓ refactor: remover pastas nao usadas da solution #51

Summary

All jobs

- ✓ Build, Test and Coverage
- ✓ Trivy Scan
- ✓ OWASP Dependency-Check Scan
- ✓ SonarQube Analysis

Run details

- Usage
- Workflow file

Triggered via push 3 days ago

GuiToniello pushed -> 1d51248 main

Status: Success

Total duration: 13m 20s

Artifacts: 4

build.yml

on: push

Build, Test and Coverage 4m 51s

Trivy Scan 4m 9s

OWASP Dependency-Ch... 4m 13s

SonarQube Analysis 4m 0s

Você pode conferir a versão ampliada no GitHub ([link](#))

## Artifacts

Produced during runtime

| Name                                                                                                      | Size    | Digest                                                                    |                                                                                     |                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------|---------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  coverage-report         | 586 KB  | sha256: c0ea5b410e71445b33ee4561db419f42fa4a73d2e04ba7aabffc9aa792683b... |  |   |
|  dependency-check-report | 156 KB  | sha256: f6fcef0d7af335b747027cf63482dd40f9c605d1c6c548a2e898adb35d86fd... |  |   |
|  test-results            | 276 KB  | sha256: f0e9885b178dd523b5ed426b0e5bc08a2f7f70a67a0ce17b69cebfbe09aa35... |  |   |
|  trivy-report            | 15.3 KB | sha256: 52413ac149d2e1713b416c9cf05388b603faef952eb2eb6bebcad499665682... |  |   |

Você pode conferir a versão ampliada no GitHub ([link](#))

SonarQube

cloud

Favorite projects

Assigned issues

Explore

Upgrade

Filters

Quality Gate

✓ Passed

1

✗ Failed

0

Reliability

A

1

B

0

C

0

D

0

E

0

Security

A

1

B

0

C

0

D

0

E

0

Security Review

Favorite Projects

Projects you marked as favorites

Search projects

Perspective

Overall Status

Sort by

Name

1 projects

★

Guilherme Toniello / tech-challenge-fase-1-soat16-rm374658

Public

✓ Passed

Last analysis: 12/06/2026, 17:24 • 3.8k Lines of Code • C#, Docker

A 0

A 0

A 0

A 100%

86.8%

0.6%

Security

Reliability

Maintainability

Hotspots Reviewed

Coverage

Duplications

1 of 1 shown

© 2018-2026 SonarSource Sàrl. All rights reserved.

Terms

Pricing

Privacy

Cookie Policy

Security

Community

Documentation

Contact us

Status

About

Você pode conferir a versão ampliada no SonarQube ([link](#))

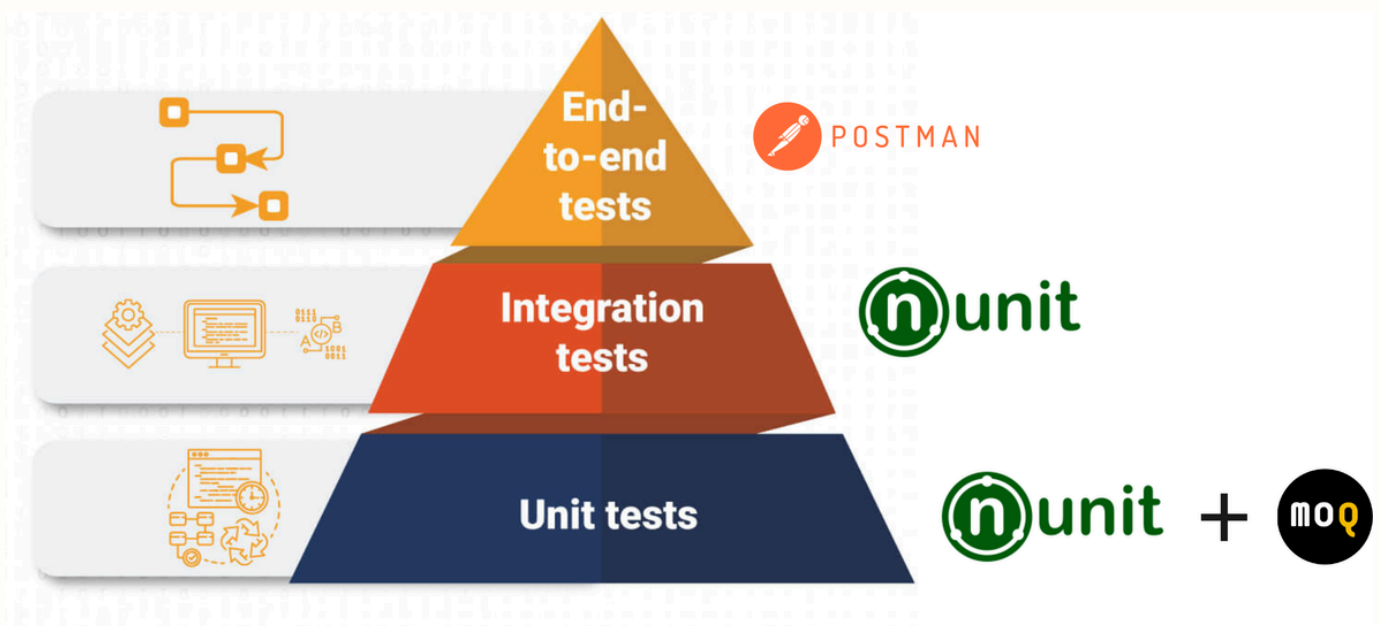
# TESTES AUTOMATIZADOS

Testes automatizados são fundamentais para o ciclo de vida da maioria dos produtos de software.

Esses testes garantem que o sistema funciona da forma esperada, validando entradas e saídas do sistema em diferentes níveis e escala.

Assim, temos testes unitários, testes de integração e testes end-to-end (e2e) que auxiliam a equipe técnica, provendo mais confiança ao realizar modificações no código-fonte.

Dito isso, a estratégia de testes automatizados utilizada nesse projeto está representada na imagem abaixo.



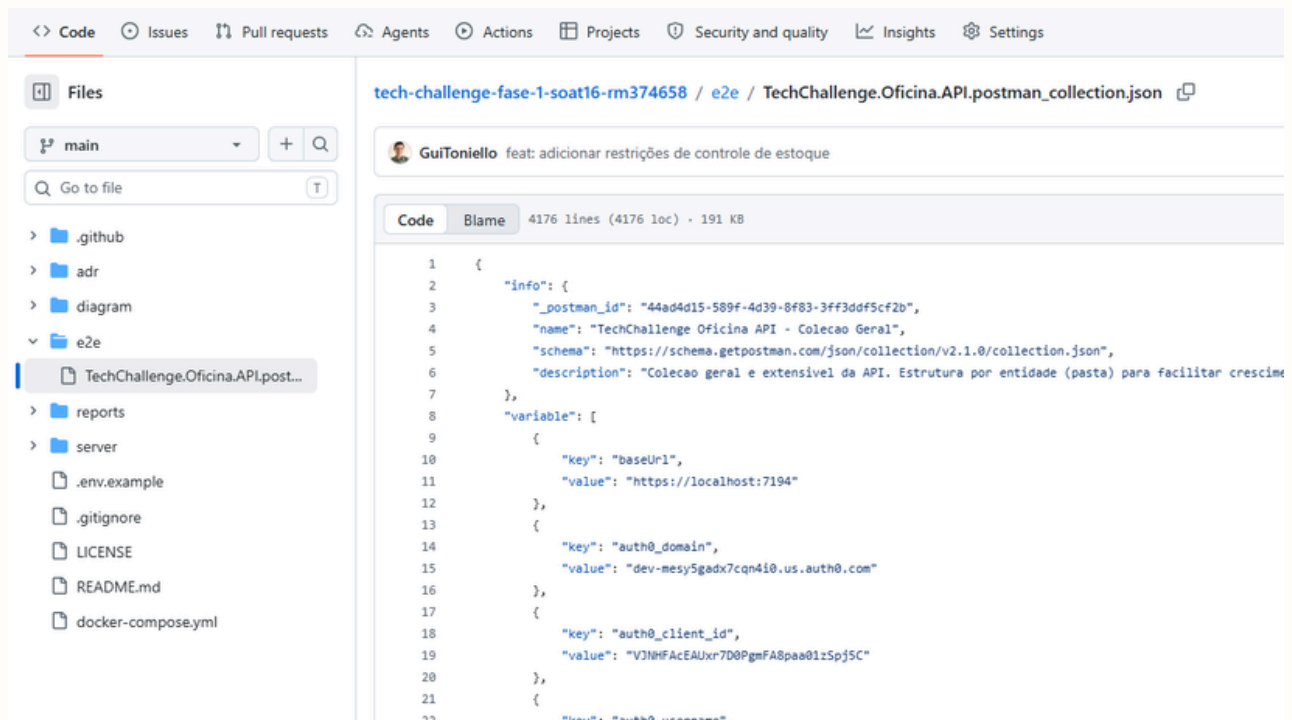
Assim, na camada mais básica, usaremos nunit com moq para criar nossos testes unitários, seguindo os **princípios do F.I.R.S.T** e também o padrão **Triple A**.

Indo adiante, para os testes de integração, vamos usar apenas o nunit para testar o produto desde a chamada na API até o Repository. Aqui, usaremos o EF InMemory para acelerar a execução de nossos testes de integração.

Por fim, usaremos o **Postman** como ferramenta de testes e2e.

O Postman é muito mais que uma simples ferramenta de chamada HTTP: ele é uma plataforma completa para testes de APIs.

Sendo assim, criamos uma collection completa, com +199 testes e2e, que pode ser acessada [nesse link](#).



Com isso, fechamos nossa pirâmide de testes: temos testes unitários, testes de integração e testes e2e.

Assim, a cada modificação, podemos executar diferentes níveis de testes - inclusive integrados aos pipelines de CI - para garantir que não adicionamos bugs ou comportamentos inesperados.

Dessa forma, nosso sistema irá crescer de forma saudável, quando associado com técnicas como TDD.

Nas próximas páginas dessa seção, você confere as imagens dos testes em diferentes níveis.



0 Warnings 0 Errors

Run | Debug | Profile

TechChallenge.Oficina.Infra.Email.Tests

🕒 Total Duration: 209 ms

7 Passed

## Git Changes

↑↓ 0/0 ^  1  main ^  tech-challenge-fase-1-soat16 ^  1



**PASS** Email atualizado

# IMPLEMENTAÇÃO

Agora, chegou a hora de explorarmos a implementação do sistema.

o código-fonte é a expressão de todas as regras e estratégias até aqui comentadas.

Você pode explorar o código fonte usando o repositório do GitHub:

<https://github.com/GuiToniello/tech-challenge-fase-1-soat16-rm374658/tree/main/server>

Na pasta `server/`, você encontra todo o código fonte implementado em dotnet 10, a ultima versão LTS até o momento da escrita desse relatório.

Todas as instruções de “como rodar o ambiente” e “como testar” estão no `readme`.

Recomenda-se usar a collection do POSTMAN, na pasta `e2e`, nela já está preparada para obter o JWT e fazer as chamadas HTTP.

Fique à vontade para explorar o projeto e entender como concretizei todas as regras e estratégias até então debatidas nesse relatório.

O código é criado no modelo iterativo e incremental, usando também **conventional commits**, para descrever corretamente as alterações realizadas no sistema.

Não deixe de conferir também as GitHub Actions, elas contém toda a concretização da estratégia de CI que foi apresentado anteriormente.

A seguir, será apresentado documentos adicionais, como o dicionário de linguagem ubíqua, links para ADRs e anexos.

# DICIONÁRIO LINGUAGEM ÚBIQUA

|    |                         |                                                                                                                                                                           |
|----|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Cliente                 | Pessoa ou organizacao que solicita atendimento da oficina, informa os problemas enfrentados no veiculo, acompanha o andamento da OS e aprova ou reprova o orcamento.      |
| 2  | Atendente               | Colaborador responsavel pelo atendimento inicial ao cliente, cadastro das informacoes necessarias, abertura da Ordem de Servico e entrega final da OS/veiculo ao cliente. |
| 3  | Tecnico                 | Colaborador responsavel por diagnosticar o problema do veiculo, criar o laudo, executar os servicos da Ordem de Servico e finalizar a execucao tecnica.                   |
| 4  | Estoquista              | Colaborador responsavel por cadastrar insumos, receber solicitacoes de separacao, separar insumos e criar ordem de compra quando o insumo nao esta disponivel.            |
| 5  | Ordem de Servico        | Registro central do atendimento da oficina. Reune cliente, veiculo, problema relatado, servicos solicitados, laudo, orcamento, insumos, status e informacoes da execucao. |
| 6  | Veiculo                 | Bem do cliente atendido pela oficina. Deve possuir informacoes como placa, marca, modelo e ano.                                                                           |
| 7  | Problema Relatado       | Descricao inicial feita pelo cliente sobre os problemas enfrentados no veiculo. Serve como base para a abertura da OS e posterior diagnostico tecnico.                    |
| 8  | Servico Solicitado      | Servico inicialmente informado ou solicitado na abertura da OS, como troca de oleo, alinhamento ou outro atendimento mecanico.                                            |
| 9  | Laudo                   | Documento/registro tecnico criado apos o diagnostico. Contem a causa do problema relatado, a solucao sugerida, servicos, pecas/insumos e horas estimadas.                 |
| 10 | Diagnostico             | Avaliacao tecnica realizada pelos tecnicos para identificar a causa do problema da OS. O diagnostico resulta na criacao do laudo.                                         |
| 11 | Orcamento               | Estimativa de custos gerada com base no laudo, considerando servicos, pecas/insumos, horas estimadas, valores unitarios e valores totais.                                 |
| 12 | Orcamento Detalhado     | Modelo de leitura do orcamento com informacoes como pecas usadas/orcadas, horas estimadas, valores unitarios e valores totais.                                            |
| 13 | Aprovacao do Orcamento  | Acao do cliente que autoriza a execucao dos servicos e o uso dos insumos previstos no orcamento.                                                                          |
| 14 | Reprovacao do Orcamento | Acao do cliente que nao autoriza a execucao do orcamento. Na modelagem, a reprovacao encerra a Ordem de Servico.                                                          |
| 15 | Status da OS            | Situacao atual da Ordem de Servico no fluxo. Os requisitos citam status como Recebida, Em diagnostico, Aguardando aprovacao, Em execucao, Finalizada e Entregue.          |
| 16 | OS Recebida             | Estado/evento que indica que a Ordem de Servico foi aberta e recebida pelo sistema/oficina para continuidade do atendimento.                                              |
| 17 | OS Em Diagnostico       | Estado em que a Ordem de Servico esta sendo avaliada pelos tecnicos para identificar o problema e gerar o laudo.                                                          |
| 18 | OS Aguardando Aprovacao | Estado em que o orcamento ja foi enviado ao cliente e o sistema aguarda a aprovacao ou reprovacao.                                                                        |
| 19 | OS Em Execucao          | Estado em que a Ordem de Servico aprovada esta sendo executada pelos tecnicos.                                                                                            |
| 20 | OS Finalizada           | Estado/evento que indica que os tecnicos finalizaram a execucao dos servicos da Ordem de Servico.                                                                         |
| 21 | OS Entregue             | Estado/evento que indica que o atendente realizou a entrega da Ordem de Servico e do veiculo ao cliente.                                                                  |
| 22 | OS Encerrada            | Estado/evento usado quando o ciclo da OS termina, por exemplo apos a reprovacao do orcamento ou conclusao administrativa do fluxo.                                        |
| 23 | Insumo                  | Item usado ou consumido na execucao de uma Ordem de Servico. Na modelagem, possui informacoes como nome, quantidade, fabricante, modelo e preco medio.                    |
| 24 | Peca                    | Tipo de insumo necessario para execucao de servicos na oficina. Pode ser incluido no orcamento e separado para uso na OS.                                                 |
| 25 | Estoque                 | Controle de disponibilidade de pecas e insumos da oficina. Apoia a separacao, uso, consulta de disponibilidade e controle de entrada/saida de insumos.                    |
| 26 | Ficha de Inventario     | Registro com dados do insumo, quantidade, preco medio e demais informacoes usadas no controle de estoque.                                                                 |
| 27 | Separacao de Insumos    | Processo em que os insumos solicitados para uma OS sao separados pelo estoquista para que os tecnicos possam executar o servico.                                          |
| 28 | Ordem de Separacao      | Registro criado para solicitar e controlar a separacao de insumos necessarios para a execucao de uma Ordem de Servico.                                                    |
| 29 | Insumo Separado         | Evento que indica que o estoquista separou o insumo solicitado para a execucao da OS.                                                                                     |
| 30 | Insumo Disponivel       | Evento/condicao que indica que o insumo necessario existe em estoque e pode ser separado para a OS.                                                                       |

|    |                                |                                                                                                                                                                                                    |
|----|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31 | Insumo Nao Disponivel          | Evento/condicao que indica que o insumo necessario nao esta disponivel em estoque, exigindo compra.                                                                                                |
| 32 | Compra de Insumos              | Fluxo alternativo acionado quando o insumo necessario nao esta disponivel em estoque. Envolve criacao de ordem de compra e integracao com SCM.                                                     |
| 33 | Ordem de Compra                | Registro criado pelo estoquista quando e necessario comprar insumos indisponiveis.                                                                                                                 |
| 34 | SCM                            | Sistema externo de Gerenciamento da Cadeia de Suprimentos usado no fluxo de compra de insumos.                                                                                                     |
| 35 | Notificacao                    | Aviso enviado a atores do sistema quando ocorre uma acao relevante, como chegada de nova OS, aprovacao do orcamento ou necessidade de continuidade do fluxo.                                       |
| 36 | Provedor de Email              | Sistema externo usado para enviar comunicacoes, como o envio do orcamento ao cliente.                                                                                                              |
| 37 | WhatsApp                       | Canal/sistema externo de mensageria usado como possibilidade de envio de notificacoes ou comunicacoes ao cliente.                                                                                  |
| 38 | Sendgrid / Resend              | Provedores externos de email representados na modelagem para apoio ao contexto de Notificacoes e Avisos.                                                                                           |
| 39 | Auth0 / Provedor de Identidade | Sistema externo associado ao contexto de Identidade e Acesso, responsavel por login, identificacao do usuario e emissao de tokens.                                                                 |
| 40 | Identidade e Acesso            | Contexto responsavel por autenticacao, autorizacao e controle de acesso dos usuarios ao sistema.                                                                                                   |
| 41 | Notificacoes e Avisos          | Contexto responsavel por enviar comunicacoes para clientes, tecnicos, atendentes ou estoquistas por canais externos.                                                                               |
| 42 | Gerenciamento de Clientes      | Contexto responsavel por cadastro de clientes e veiculos, incluindo informacoes como CPF/CNPJ, contato, placa, marca, modelo e ano.                                                                |
| 43 | Ordem de Servico               | Contexto delimitado responsavel pelo ciclo da OS, incluindo abertura, acompanhamento, execucao, finalizacao e entrega.                                                                             |
| 44 | Orcamento                      | Contexto delimitado responsavel por gerar, enviar, aprovar ou reprovar orcamentos vinculados a uma OS.                                                                                             |
| 45 | Gerenciamento de Insumos       | Contexto responsavel pelo cadastro, consulta de disponibilidade, controle de estoque e relatorios de entrada/saida de insumos.                                                                     |
| 46 | Separacao de Insumos           | Contexto responsavel por controlar ordens de separacao pendentes e concluidas, alem da separacao de insumos para OS.                                                                               |
| 47 | Compra de Insumos              | Contexto responsavel por ordem de compra, cotacao e acompanhamento da compra de insumos quando nao ha disponibilidade em estoque.                                                                  |
| 48 | Modelo de Leitura              | Representacao de informacoes consultadas no fluxo, como Ordem de Servico Completa, Orcamento Detalhado, Informacoes do Orcamento Aprovado e Detalhes da OS Finalizada.                             |
| 49 | Politica                       | Regra que reage a eventos e dispara comandos ou mudancas no fluxo. Exemplo: quando o laudo e criado, o sistema gera o orcamento; quando o cliente aprova o orcamento, os tecnicos sao notificados. |
| 50 | Comando                        | Intencao ou acao executada por um ator ou sistema, como Abrir Ordem de Servico, Criar Laudo, Gerar Orcamento, Enviar Orcamento, Aprovar Orcamento ou Separar Insumos.                              |
| 51 | Evento                         | Fato relevante que ja aconteceu no dominio, como Problema Relatado, Cliente Cadastrado, Laudo Criado, Orcamento Enviado, Orcamento Aprovado, Insumo Separado ou OS Entregue.                       |
| 52 | Evento Pivotal                 | Marco importante do fluxo usado para organizar o Event Storming em etapas, como Abertura da OS, Orcamento da OS, Execucao da OS, Finalizacao da OS, Registro de Insumos, Separacao, Uso e Compra.  |
| 53 | Ponto de Atencao               | Pergunta, duvida ou incerteza identificada durante a modelagem, como quais informacoes do cliente cadastrar, qual nivel de detalhamento do orcamento ou como formalizar a aprovacao.               |

# Eventos Principais da Linguagem Ubiqua



Evento

Significado

|       |                                                                                                                                                                                           |                                                                             |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1     | Problema Relatado                                                                                                                                                                         | O cliente informou os problemas enfrentados no veículo.                     |
| 2     | Cliente Cadastrado                                                                                                                                                                        | As informacoes cadastrais do cliente foram registradas.                     |
| 3     | Ordem de Servico Recebida                                                                                                                                                                 | A OS foi aberta e recebida para atendimento.                                |
| 4     | Problema Diagnosticado                                                                                                                                                                    | Os tecnicos diagnosticaram o problema da OS.                                |
| 5     | Laudo Criado                                                                                                                                                                              | O tecnico criou o laudo com causa, solucao sugerida e informacoes tecnicas. |
| 6     | Orcamento Gerado                                                                                                                                                                          | O sistema gerou o orcamento com base no laudo.                              |
| 7     | Orcamento Enviado                                                                                                                                                                         | O orcamento foi enviado ao cliente.                                         |
| 8     | Orcamento Aprovado                                                                                                                                                                        | O cliente aprovou o orcamento.                                              |
| 9     | Orcamento Reprovado                                                                                                                                                                       | O cliente reprovou o orcamento.                                             |
| 10    | Insumos Separados                                                                                                                                                                         | Os insumos necessarios foram separados para a execucao da OS.               |
| 11    | Ordem de Servico Em...                                                                                                                                                                    | A OS entrou em execucao pelos tecnicos.                                     |
| 12    | Ordem de Servico Finalizada                                                                                                                                                               | A execucao tecnica da OS foi finalizada.                                    |
| 13    | Ordem de Servico Entregue                                                                                                                                                                 | A OS e o veiculo foram entregues ao cliente.                                |
| 14    | Ordem de Servico Encerrada                                                                                                                                                                | O ciclo da OS foi encerrado.                                                |
| 15    | Insumo Cadastrado                                                                                                                                                                         | O insumo foi registrado no sistema.                                         |
| 16    | Ordem de Separacao Criada                                                                                                                                                                 | Foi criada uma ordem para separar insumos para uma OS.                      |
| +     |   Insumo Disponivel | O insumo solicitado existe em estoque.                                      |
| 18    | Insumo Separado                                                                                                                                                                           | O estoquista separou o insumo solicitado.                                   |
| 19    | Ordem de Compra Criada                                                                                                                                                                    | Foi criada uma ordem de compra para insumo indisponivel.                    |
| 20    | Insumo Utilizado                                                                                                                                                                          | O insumo foi utilizado na execucao do servico.                              |
| <hr/> |                                                                                                                                                                                           |                                                                             |
| +     |                                                                                                                                                                                           |                                                                             |

## Regras / Politicas do Dominio

Politica

Descricao

1

Laudo gera orcamento

Quando o laudo e criado, o sistema gera o orcamento com base nas informacoes do laudo.

2

Envio exige aprovacao

Apos o envio do orcamento, a OS deve aguardar aprovacao do cliente.

3

Aprovacao notifica tecnicos

Quando o cliente aprova o orcamento, os tecnicos sao notificados da aprovacao.

4

Aprovacao permite separacao de insumos

Quando os tecnicos sao notificados da aprovacao, os insumos necessarios devem ser separados.

5

Insumos separados liberam execucao

Quando os insumos sao separados, os tecnicos podem iniciar a execucao da OS.

6

Reprovacao encerra OS

Quando o cliente reprova o orcamento, a Ordem de Servico e encerrada.

7

Indisponibilidade exige compra

Quando nao ha insumo disponivel, e necessario criar uma ordem de compra.

8

Compra reinicia fluxo de separacao

Quando o SCM notifica que o insumo foi comprado e esta disponivel, o fluxo retorna para cadastro/solicitacao/separacao do insumo.

# Comandos Principais da Linguagem Ubiqua

| <div><div><div>↺</div><div>📅</div><div>📄</div><div>🔍</div><div>↕</div><div>🔗</div><div>⚙️</div><div>⬇️</div><div>🔍</div></div></div> |                               |                                     |
|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|-------------------------------------|
|                                                                                                                                      | 📄 Comando                     | ☰ Ator principal                    |
| 1                                                                                                                                    | Relatar Problema              | Cliente                             |
| 2                                                                                                                                    | Cadastrar Cliente             | Atendente                           |
| 3                                                                                                                                    | Abrir Ordem de Servico        | Atendente                           |
| 4                                                                                                                                    | Notificar Tecnicos            | Sistema                             |
| 5                                                                                                                                    | Diagnosticar Problema         | Tecnicos                            |
| 6                                                                                                                                    | Criar Laudo                   | Tecnicos                            |
| 7                                                                                                                                    | Gerar Orcamento               | Sistema                             |
| 8                                                                                                                                    | Enviar Orcamento              | Tecnicos / Sistema                  |
| 9                                                                                                                                    | Aprovar Orcamento             | Cliente                             |
| 10                                                                                                                                   | Reprovar Orcamento            | Cliente                             |
| 11                                                                                                                                   | Separar Insumos               | Estoquista / Tecnico conforme fluxo |
| 12                                                                                                                                   | Executar Servico              | Tecnicos                            |
| 13                                                                                                                                   | Finalizar Servico             | Tecnicos                            |
| 14                                                                                                                                   | Notificar Atendente           | Tecnicos / Sistema                  |
| 15                                                                                                                                   | Realizar Entrega da OS        | Atendente                           |
| 16                                                                                                                                   | Cadastrar Insumo              | Estoquista                          |
| 17                                                                                                                                   | Solicitar Separacao de Insumo | Tecnico                             |
| 18                                                                                                                                   | Criar Ordem de Compra         | Estoquista                          |
| +                                                                                                                                    |                               |                                     |

# ARCHITECTURAL DECISION RECORDS(ADR)

| ADR | Link                                                                            |
|-----|---------------------------------------------------------------------------------|
| -   | <u><a href="#">INDICE</a></u>                                                   |
| 001 | <u><a href="#">Arquitetura em Camadas</a></u>                                   |
| 002 | <u><a href="#">Domain-Driven Design e Entidades Ricas</a></u>                   |
| 003 | <u><a href="#">Value Objects para Validação de Identificação (CPF/CNPJ)</a></u> |
| 004 | <u><a href="#">Repository Pattern e Persistência</a></u>                        |
| 005 | <u><a href="#">Separação de Contratos em Commands/Queries/ViewModels</a></u>    |
| 006 | <u><a href="#">Mapeamento com AutoMapper na Camada Application</a></u>          |
| 007 | <u><a href="#">EntityFramework Core com PostgreSQL</a></u>                      |
| 008 | <u><a href="#">Swagger para Documentação de API</a></u>                         |
| 009 | <u><a href="#">Migrações Automáticas no Startup</a></u>                         |
| 011 | <u><a href="#">Envio de E-mail de Orçamento com Resend</a></u>                  |
| 012 | <u><a href="#">Controle de Estoque de Insumos</a></u>                           |

# REFERÊNCIAS

As referências abaixo foram usadas como base para a implementação dos padrões descritos na seção "arquitetura".

## **Livros e Artigos**

Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). Pattern-Oriented Software Architecture: A System of Patterns. FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2003.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley, 1994.

MARTIN, Robert C. Agile Software Development, Principles, Patterns, and Practices. Upper Saddle River: Prentice Hall, 2002.

WIRFS-BROCK, Rebecca J. Toward Exception-Handling Best Practices and Patterns. IEEE Software, v. 23, n. 5, p. 11-13, 2006. DOI: 10.1109/MS.2006.129.

## **Material de Apoio**

FOWLER, Martin. CQRS. Martin Fowler, 14 jul. 2011. Disponível em: <https://martinfowler.com/bliki/CQRS.html>. Acesso em: 10 jun. 2026.

STAFFORD, Randy. Service Layer. Martin Fowler, 5 mar. 2003. Disponível em: <https://martinfowler.com/eaCatalog/serviceLayer.html>. Acesso em: 12 jun. 2026.

FOWLER, Martin. Inversion of Control Containers and the Dependency Injection pattern. Martin Fowler, 23 jan. 2004. Disponível em: <https://martinfowler.com/articles/injection.html>. Acesso em: 02 jun. 2026.

# ANEXOS

Nessa seção, você encontra os arquivos anexados nesse relatório.

As próximas páginas apresentam:

- o relatório de vulnerabilidades do OWASP ZAP
- o relatório de análise do Trivy